

Multi-View Neural Network for Musculoskeletal Disorder Detection in Radiographic Imaging

Camille Trouvé¹, Ceren Dinc¹, Esteban Boulanger¹ Roger El Achkar¹

¹*Faculté de Gestion Economie et Sciences - Ecole du Numérique*

January 11, 2026

Abstract

Musculoskeletal disorders are among the main causes of disability, according to the World Health Organization. This study aims to develop a tool capable of determining abnormalities in musculoskeletal radiographs. The proposed model is a transformer-based deep neural network with a multiple-view fusion module to integrate multiple radiographic projections, mimicking how radiologists interpret radiographic views. The model was trained and tested on the MURA dataset (Stanford), a large open-access dataset combining 40,561 radiographic images of the upper extremity from multiple studies. The model achieved an accuracy of XX (to be determined). The results are expected to demonstrate the benefit of multi-view integration for improving diagnostic performance and robustness in musculoskeletal abnormality detection.

1 Introduction

Write your introduction here.

2 Material and Method

2.1 Dataset

For this project, we worked with the MURA dataset (Musculoskeletal Radiographs) released by the Stanford Machine Learning Group. It contains upper-limb radiographs labeled as normal or abnormal at the study level. In total, the dataset includes 40 561 images grouped into 14 656 studies. We followed the official split provided by the authors, which gave us 36 808 images for training (13 457 studies) and 3 197 images for validation (1 199 studies).

A study corresponds to a single patient examination and can contain between one and four radiographic views depending on the body part examined (for example: elbow, finger, forearm, hand, humerus, shoulder or wrist). Since the goal is binary classification (normal vs abnormal), each study has a single label, even if it contains several images. The dataset is also slightly imbalanced, with around 38.5% abnormal studies.

To formalize the data, we write:

$$\mathcal{D} = \{(S_i, y_i)\}_{i=1}^N, \tag{1}$$

where $S_i = \{x_i^{(v)}\}_{v=1}^{n_i}$ is the i -th study, $n_i \in \{1, 2, 3, 4\}$ is the number of views available, and $y_i \in \{0, 1\}$ is the study-level label.

We implemented a custom PyTorch dataset class (`data/dataset.py`) to load the studies, handle the fact that the number of views varies, and return both the images and a mask indicating which views are present.

2.2 Preprocessing and Data Augmentation

Before being passed to the model, all images were converted to three channels and resized to 224×224 pixels so that they match the input size expected by the Vision Transformer. We also normalized the pixel intensities using the standard ImageNet normalization.

During training, we applied several data augmentation steps (defined in `data/transforms.py`) such as random horizontal flips, small rotations and slight contrast adjustments. These augmentations help the model generalize better because radiographs can vary a lot in terms of acquisition conditions. For a study S_i , each image is transformed as:

$$\tilde{x}_i^{(v)} = T(x_i^{(v)}), \quad v = 1, \dots, n_i. \quad (2)$$

Because studies do not always contain four views, we padded missing views to obtain a fixed-size tensor of shape $[4, 3, 224, 224]$. We also created a binary mask $m_i \in \{0, 1\}^4$ indicating which views are real and which are padding.

2.3 Model Architecture

The model is based on a multi-view classification approach. The intuition is that radiologists rarely rely on a single projection when evaluating an X-ray, so our model should be able to combine several views at once. The architecture is composed of three main parts: (1) a shared Vision Transformer backbone, (2) a fusion module based on cross-attention, and (3) a final classification layer.

2.3.1 Vision Transformer Backbone

Each view is processed independently with the same Vision Transformer (ViT-Base Patch16-224), implemented in `models/vit_backbone.py`. The model was initialized with ImageNet pretraining. For each view $\tilde{x}_i^{(v)}$, the backbone produces a feature vector of size 768 using the class token:

$$h_i^{(v)} = f_\theta(\tilde{x}_i^{(v)}) \in \mathbb{R}^{768}. \quad (3)$$

For each study, we collect the feature vectors from all available views and stack them into a matrix:

$$H_i = \begin{bmatrix} h_i^{(1)} \\ \vdots \\ h_i^{(V_{\max})} \end{bmatrix} \in \mathbb{R}^{V_{\max} \times d}, \quad (4)$$

with padded rows for missing views. The same backbone is used for every image in the dataset.

2.3.2 Cross-Attention Multi-View Fusion

Since the number of images per study varies, we use a cross-attention module (`models/multiview_fusion`) to fuse the view embeddings into a single representation. The idea is that the model can learn to focus more on informative views and ignore padded ones.

We introduce Q learnable query tokens $Q_0 \in \mathbb{R}^{Q \times d}$ and compute queries, keys and values as:

$$Q_i = Q_0 W_Q, \quad (5)$$

$$K_i = H_i W_K, \quad (6)$$

$$V_i = H_i W_V. \quad (7)$$

We then apply attention over the views, while masking padded positions:

$$A_i = \text{softmax} \left(\frac{Q_i K_i^\top}{\sqrt{d_k}} + M_i \right), \quad (8)$$

and derive the fused representation:

$$U_i = A_i V_i. \quad (9)$$

When $Q = 1$, the result is directly a single feature vector; otherwise, we average the Q outputs:

$$z_i = \frac{1}{Q} \sum_{q=1}^Q U_i^{(q)} W_O. \quad (10)$$

2.3.3 Classification Head

The final representation z_i is passed through a dropout layer and a fully connected layer followed by a sigmoid activation:

$$\hat{y}_i = \sigma(w^\top \text{Dropout}(z_i) + b), \quad (11)$$

where $\hat{y}_i \in (0, 1)$ is the predicted probability that the study is abnormal.

2.4 Training Procedure

To deal with the class imbalance, we used a weighted binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B [w_+ y_i \log(\hat{y}_i) + w_- (1 - y_i) \log(1 - \hat{y}_i)]. \quad (12)$$

Training was performed with the AdamW optimizer using a typical learning rate of 10^{-4} and weight decay 10^{-4} . The training loop (`training/trainer.py`) computes the loss, updates the weights, evaluates the model on the validation set at each epoch, and saves checkpoints using `training/checkpoint_manager.py`. Mixed precision and gradient accumulation could be enabled depending on GPU memory.

2.5 Evaluation Metrics

We evaluated the model mainly with the AUC-ROC score, since it is commonly used in medical imaging for binary classification. We also report accuracy, sensitivity and specificity. For a given threshold τ , these metrics are defined as:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \quad (13)$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (14)$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}. \quad (15)$$

AUC-ROC is computed from the ROC curve by varying τ across $[0, 1]$.

2.6 Model Interpretability

To visualize what the model focuses on, we used a Grad-CAM approach adapted for transformers (`evaluation/visualization.py`). We compute gradients of the abnormality score with respect to intermediate feature maps and use them to produce attention heatmaps:

$$\alpha_c = \frac{1}{HW} \sum_{u=1}^H \sum_{v=1}^W \frac{\partial s}{\partial F_{cuv}}, \quad (16)$$

$$L_{\text{GradCAM}} = \text{ReLU} \left(\sum_{c=1}^C \alpha_c F_c \right). \quad (17)$$

For multi-view studies, we generate one map per view, which helps assess whether the model attends to relevant anatomical regions.

3 Results

Write your results here.

4 Discussion

Write your discussion here.